

```

'use client';

import { useRef, useState } from 'react';
import ExcelJS from 'exceljs';
import { saveAs } from 'file-saver';

import { Modal } from '../../../components/ui/Modal';
import { useBulkUpload } from '../../../hooks/useCandidates';

import type { BulkUploadResult } from '../../../types/candidate.types';

interface Props {
  open: boolean;
  onClose: () => void;
}

const MAX_FILE_SIZE = 10 * 1024 * 1024;

const ACCEPTED_EXTENSIONS = ['csv', 'xlsx', 'xls'];

const HEADERS = [
  'candidate_name',
  'email',
  'phone_number',
  'gender',
  'date_of_birth',
  'current_company_name',
  'current_company_designation',
  'qualification',
  'location',
  'total_experience',
  'relevant_experience',
  'apply_department',
  'apply_designation',
  'current_salary',
  'expected_salary',
  'notice_period',
  'immediate_joiner',
  'expected_joining_date',
  'own_vehicle',
  'source',
  'reference_source',
  'remarks',
];

const SAMPLE_ROW = [

```

```

    'Priya Sharma',
    'priya@gmail.com',
    '+919876543210',
    'Female',
    '1995-05-15',
    'Infosys',
    'Senior Engineer',
    'B.E. Computer Science',
    'Bengaluru',
    5,
    4,
    'IT',
    'Sr. Software Engineer',
    75000,
    90000,
    30,
    'false',
    '2026-07-01',
    'true',
    'Naukri',
    'Job Portal',
    'Strong profile',
  ];

const DROPZONE_STYLE: React.CSSProperties = {
  borderRadius: 'var(--r2)',
  padding: '32px 24px',
  textAlign: 'center',
  cursor: 'pointer',
  transition: 'all .15s',
  marginBottom: 16,
};

export function BulkUploadModal({
  open,
  onClose,
}: Props) {
  const fileRef = useRef<HTMLInputElement>(null);

  const [file, setFile] = useState<File | null>(null);

  const [result, setResult] =
    useState<BulkUploadResult | null>(null);

  const [dragOver, setDragOver] =
    useState(false);

  const [error, setError] =

```

```

    useState<string | null>(null);

    const bulkMutation = useBulkUpload();

    const resetState = () => {
        setFile(null);
        setResult(null);
        setError(null);
    };

    const handleClose = () => {
        resetState();
        onClose();
    };

    const validateFile = (f: File) => {
        const extension =
            f.name.split('.').pop()?.toLowerCase();

        if (
            !extension ||
            !ACCEPTED_EXTENSIONS.includes(extension)
        ) {
            return 'Please upload CSV, XLSX, or XLS file';
        }

        if (f.size > MAX_FILE_SIZE) {
            return 'Maximum file size is 10 MB';
        }

        return null;
    };

    const handleFile = (f: File | null) => {
        if (!f) return;

        const validationError = validateFile(f);

        if (validationError) {
            setError(validationError);
            return;
        }

        setError(null);
        setFile(f);
        setResult(null);
    };

```

```

const handleUpload = async () => {
  if (!file || bulkMutation.isPending) return;

  try {
    const res =
      await bulkMutation.mutateAsync(file);

    setResult(res.data);
  } catch (err: any) {
    setError(
      err?.response?.data?.message ||
        'Upload failed',
    );
  }
};

const applyDropdownValidation = (
  sheet: ExcelJS.Worksheet,
  column: string,
  values: string[],
) => {
  for (let i = 2; i <= 500; i++) {
    sheet.getCell(`${column}${i}`).dataValidation = {
      type: 'list',
      allowBlank: true,
      formulae: [`"${values.join(',')}"`],
    };
  }
};

const downloadTemplate = async () => {
  const workbook = new ExcelJS.Workbook();

  const sheet =
    workbook.addWorksheet('Candidates');

  sheet.addRow(HEADERS);

  sheet.addRow(SAMPLE_ROW);

  // Header style
  const headerRow = sheet.getRow(1);

  headerRow.font = {
    bold: true,
    color: { argb: 'FFFFFF' },
  };
};

```

```

headerRow.fill = {
  type: 'pattern',
  pattern: 'solid',
  fgColor: { argb: '2563EB' },
};

headerRow.alignment = {
  vertical: 'middle',
  horizontal: 'center',
};

// Column widths
HEADERS.forEach((header, i) => {
  const width =
    Math.max(header.length + 5, 20);

  sheet.getColumn(i + 1).width = width;
});

// Date formatting
sheet.getColumn(5).numFmt =
  'yyyy-mm-dd';

sheet.getColumn(18).numFmt =
  'yyyy-mm-dd';

// Freeze header
sheet.views = [
  {
    state: 'frozen',
    ySplit: 1,
  },
];

// Auto filter
sheet.autoFilter = {
  from: 'A1',
  to: 'V1',
};

// Dropdown validations
applyDropdownValidation(
  sheet,
  'D',
  [
    'Male',
    'Female',
    'Other',
  ]
);

```

```

        'Prefer not to say',
    ],
);

applyDropdownValidation(
    sheet,
    'Q',
    ['true', 'false'],
);

applyDropdownValidation(
    sheet,
    'S',
    ['true', 'false'],
);

applyDropdownValidation(
    sheet,
    'T',
    [
        'Naukri',
        'LinkedIn',
        'CollarCheck',
        'Referral',
        'Walk-in',
        'Indeed',
        'Direct',
        'Other',
    ],
);

const buffer =
    await workbook.xlsx.writeBuffer();

saveAs(
    new Blob([buffer]),
    'nexhr_candidates_template.xlsx',
);
};

const downloadErrorReport = () => {
    if (!result?.errors.length) return;

    const csvRows = [
        'Row,Candidate Name,Error Reason',

        ...result.errors.map(
            e =>

```

```

        `${e.row},${e.name}","${e.reason}"`,
    ),
];

const blob = new Blob(
    [csvRows.join('\n')],
    { type: 'text/csv' },
);

const url =
    URL.createObjectURL(blob);

const a =
    document.createElement('a');

a.href = url;

a.download =
    'bulk_upload_errors.csv';

a.click();

URL.revokeObjectURL(url);
};

return (
    <Modal
        open={open}
        onClose={handleClose}
        title="Bulk Upload Candidates"
        subtitle="Import multiple candidates from CSV or Excel"
        width={580}
        footer={
            result ? (
                <div
                    style={{
                        display: 'flex',
                        gap: 8,
                        justifyContent: 'flex-end',
                        width: '100%',
                    }}
                >
                    {result.errors.length > 0 && (
                        <button
                            className="btn btn-sec btn-sm"
                            onClick={downloadErrorReport}
                        >

```

```

         Download Error Report
    </button>
  )}

  <button
    className="btn btn-pri"
    onClick={handleClose}
  >
    Done
  </button>
</div>
) : (
  <>
    <button
      className="btn btn-sec"
      onClick={handleClose}
    >
      Cancel
    </button>

    <button
      className="btn btn-pri"
      onClick={handleUpload}
      disabled={
        !file ||
        bulkMutation.isPending
      }
      style={{
        display: 'flex',
        alignItems: 'center',
        gap: 6,
      }}
    >
      {bulkMutation.isPending && (
        <span
          style={{
            width: 12,
            height: 12,
            border:
              '2px solid rgba(255,255,255,.4)',
            borderTopColor: '#fff',
            borderRadius: '50%',
            display: 'inline-block',
            animation:
              'spin .7s linear infinite',
          }}
        >
          />

```



```

    })

    {bulkMutation.isPending
      ? 'Uploading...'
      : '↑ Upload File'}
  </button>
</>
)
}
>
{!result ? (
  <>
    {/* Template */}
    <div
      style={{
        background: 'var(--blue-lt)',
        border:
          '1px solid var(--blue-md)',
        borderRadius: 'var(--r)',
        padding: '12px 14px',
        marginBottom: 16,
        display: 'flex',
        justifyContent:
          'space-between',
        alignItems: 'center',
      }}
    >
      <div>
        <div
          style={{
            fontSize: 12,
            fontWeight: 600,
            color: 'var(--blue)',
            marginBottom: 2,
          }}
        >
          Download the Excel template first
        </div>

        <div
          style={{
            fontSize: 11,
            color: 'var(--ink4)',
          }}
        >
          {HEADERS.length} columns ·
          includes a sample row

```

```

    </div>
</div>

<button
  className="btn btn-sec btn-sm"
  onClick={downloadTemplate}
>
   Template
</button>
</div>

{/* Error */}
{error && (
  <div
    style={{
      background:
        'var(--red-lt)',
      border:
        '1px solid var(--red-bd)',
      borderRadius:
        'var(--r)',
      padding:
        '10px 14px',
      marginBottom: 16,
      fontSize: 12,
      color: 'var(--red)',
    }}
  >
    {error}
  </div>
)}

{/* Validation */}
<div
  style={{
    background:
      'var(--amber-lt)',
    border:
      '1px solid var(--amber-bd)',
    borderRadius:
      'var(--r)',
    padding:
      '10px 14px',
    marginBottom: 16,
    fontSize: 11,
    color: 'var(--amber)',
  }}

```

```

>
    <strong>
      Validation rules:
    </strong>

    <br />

    ☐ email and phone_number
    must be unique per company

    <br />

    ☐ expected_joining_date
    must be today or future

    <br />

    ☐ Supported:
    CSV, XLSX, XLS

    <br />

    ☐ Maximum upload size:
    10 MB
  </div>

  { /* Dropzone */ }
  <div
    onDragOver={e => {
      e.preventDefault();
      setDragOver(true);
    }}
    onDragLeave={() =>
      setDragOver(false)
    }
    onDrop={e => {
      e.preventDefault();
      setDragOver(false);

      handleFile(
        e.dataTransfer.files[0],
      );
    }}
    onClick={() =>
      fileRef.current?.click()
    }
    style={{
      ...DROPZONE_STYLE,

```

```

border: `2px dashed ${
  dragOver
    ? 'var(--blue)'
    : file
      ? 'var(--green)'
      : 'var(--border2)'
}` ,

background: dragOver
  ? 'var(--blue-lt)'
  : file
    ? 'var(--green-lt)'
    : 'var(--surface2)',
}}
>
<input
  ref={fileRef}
  type="file"
  accept=".csv,.xlsx,.xls"
  style={{ display: 'none' }}
  onChange={e =>
    handleFile(
      e.target.files?.[0] ||
      null,
    )
  }
/>

<div
  style={{
    fontSize: 32,
    marginBottom: 10,
  }}
>
  {file ? '✅' : '📄'}
</div>

{file ? (
  <>
    <div
      style={{
        fontSize: 13,
        fontWeight: 700,
        color: 'var(--green)',
        marginBottom: 4,
      }}
    >

```

```

        {file.name}
      </div>

      <div
        style={{
          fontSize: 11,
          color: 'var(--ink4)',
        }}
      >
        {(
          file.size / 1024
        ).toFixed(1)}{' '}
        KB 
      </div>
    </>
  ) : (
    <>
      <div
        style={{
          fontSize: 13,
          fontWeight: 600,
          color: 'var(--ink)',
          marginBottom: 4,
        }}
      >
        Drop CSV or Excel file
        here
      </div>

      <div
        style={{
          fontSize: 11,
          color: 'var(--ink4)',
        }}
      >
        Max 10 MB 
        CSV/XLSX/XLS supported
      </div>
    </>
  )}
</div>
</>
) : (
  <>
    { /* Summary */ }
    <div
      style={{
        display: 'grid',

```

```

        gridTemplateColumns:
            '1fr 1fr 1fr',
        gap: 10,
        marginBottom: 16,
    }}
>
    {[
        {
            label: 'Total rows',
            value: result.total,
            color: 'var(--blue)',
        },

        {
            label: '✓ Imported',
            value: result.success,
            color: 'var(--green)',
        },

        {
            label: '✗ Failed',
            value: result.failed,
            color:
                result.failed > 0
                ? 'var(--red)'
                : 'var(--ink4)',
        },
    ]}.map(s => (
        <div
            key={s.label}
            style={{
                textAlign: 'center',
                background:
                    'var(--surface2)',
                borderRadius:
                    'var(--r)',
                padding: '12px 8px',
            }}
        >
            <div
                style={{
                    fontSize: 22,
                    fontWeight: 700,
                    fontFamily:
                        'var(--mono)',
                    color: s.color,
                    marginBottom: 4,
                }}

```

```

    >
      {s.value}
    </div>

    <div
      style={{
        fontSize: 11,
        color: 'var(--ink4)',
      }}
    >
      {s.label}
    </div>
  </div>
)}}
</div>

{result.success > 0 && (
  <div
    style={{
      background:
        'var(--green-lt)',
      border:
        '1px solid var(--green-bd)',
      borderRadius:
        'var(--r)',
      padding:
        '10px 14px',
      fontSize: 12,
      color: 'var(--green)',
      marginBottom: 12,
    }}
  >
    ☒ {result.success}
    candidate
    {result.success !== 1
      ? 's'
      : ''}{ ' '}
    imported successfully
  </div>
)}}

{result.errors.length > 0 && (
  <>
    <div
      style={{
        display: 'flex',
        justifyContent:

```

```

        'space-between',
        alignItems: 'center',
        marginBottom: 8,
    }}
>
<div
  style={{
    fontSize: 12,
    fontWeight: 700,
    color: 'var(--red)',
  }}
>
  Failed rows (
    {result.errors.length})
</div>

<button
  className="btn btn-sec btn-sm"
  onClick={
    downloadErrorReport
  }
>
   Download CSV
</button>
</div>

<div
  style={{
    maxHeight: 220,
    overflowY: 'auto',
    border:
      '1px solid var(--border)',
    borderRadius:
      'var(--r)',
  }}
>
  <table
    style={{
      width: '100%',
      borderCollapse:
        'collapse',
      fontSize: 11,
    }}
  >
    <thead>
      <tr
        style={{

```



```

        background:
            'var(--surface2)',
        }}
    >
    <th
        style={{
            padding:
                '6px 10px',
            textAlign:
                'left',
            fontWeight: 700,
            borderBottom:
                '1px solid var(--border)',
            width: 50,
        }}
    >
        Row
    </th>

    <th
        style={{
            padding:
                '6px 10px',
            textAlign:
                'left',
            fontWeight: 700,
            borderBottom:
                '1px solid var(--border)',
        }}
    >
        Name
    </th>

    <th
        style={{
            padding:
                '6px 10px',
            textAlign:
                'left',
            fontWeight: 700,
            borderBottom:
                '1px solid var(--border)',
            color:
                'var(--red)',
        }}
    >
        Error
    </th>

```

```

    </tr>
  </thead>

  <tbody>
    {result.errors.map(
      (e, i) => (
        <tr
          key={i}
          style={{
            borderBottom:
              '1px solid var(--border)',
          }}
        >
          <td
            style={{
              padding:
                '6px 10px',
              color:
                'var(--ink4)',
            }}
          >
            #{e.row}
          </td>

          <td
            style={{
              padding:
                '6px 10px',
              fontWeight: 600,
            }}
          >
            {e.name}
          </td>

          <td
            style={{
              padding:
                '6px 10px',
              color:
                'var(--red)',
            }}
          >
            {e.reason}
          </td>
        </tr>
      ),
    )}
  </tbody>

```

```

        </table>
      </div>
    </>
  })
</>
})

<style>
{
  @keyframes spin {
    to {
      transform: rotate(360deg);
    }
  }
}
</style>
</Modal>
);
}

```